



THE UNIVERSITY OF
AUCKLAND
Te Whare Wānanga o Tāmaki Makaurau
NEW ZEALAND

ELECTENG 209

Embedded Software Design

Analog to Digital Conversion

Duleepa J Thrimawithana



Learning Objectives



- What are analog and digital signals?
- Why do we want to process digital signals rather than analog signals?
- What is an analog to digital converter (ADC)?
- Why do we need ADCs?
- How do we characterize ADCs?
- Common ADC implementations
- Understanding how a successive approximation register (SAR) ADC work
- Learning to configure the ADC in the ATmega328P
- Learning to develop code to use the ADC in the ATmega328P



Lecture Quiz



- The lecture quiz is now available on Canvas
 - Quiz is available for approximately 3 hours and allows 2 attempts
 - Best of the 2 attempts taken as the final score



THE UNIVERSITY OF
AUCKLAND
Te Whare Wānanga o Tāmaki Makaurau
NEW ZEALAND

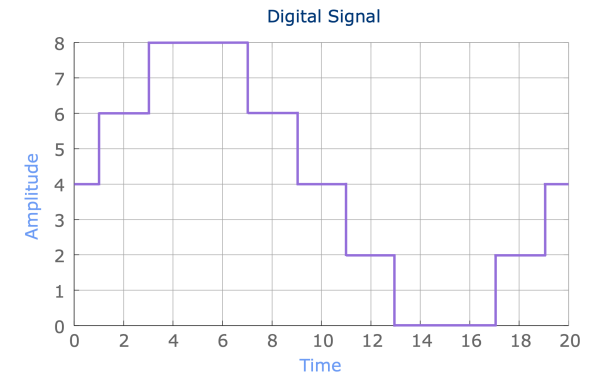
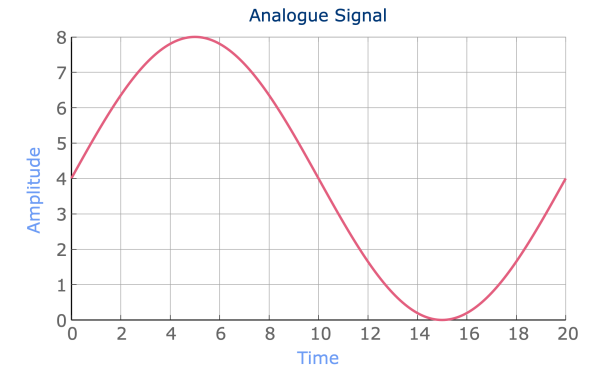
Analog and Digital Signals

Understanding the Differences



Analog vs Digital Signals

- Analog signals are continuous in both time and amplitude
 - Often sensitive to noise
 - Most physical signals are analog
 - Most electrical sensors produce analog electrical signals
 - Can be converted to a digital signal via an analog to digital converter (ADC)
- Digital signals are discrete in both time and amplitude
 - More resistant to noise
 - Cannot exactly reproduce analog signals
 - Some electrical sensors can output digital signals
 - Can be converted to an analog signal via a digital to analog converter (DAC)





Sensors and Actuators

- Often the physical signals/quantities we wish to interact with are not electrical
 - For example, temperature, light, force, torque, etc.
- An electrical transducer converts a physical signal/quantity into an electrical signal or an electrical signal into a physical signal/quantity
 - Can be categorized as electrical sensors and electrical actuators
- Electrical sensors receive a signal from a physical system, and produce an electrical signal as an output
 - For example, a temperature sensor converts temperature into an electrical signal (voltage)
- Electrical actuators receive an electrical signal as an input, and produce a physical signal as an output
 - For example, a DC motor converts an electrical signal (current) into mechanical torque



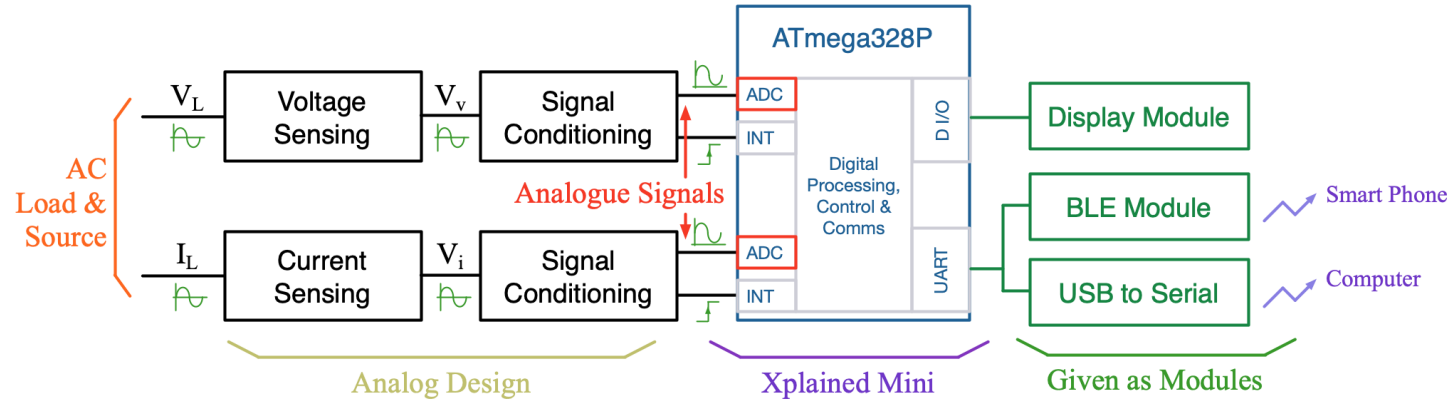


Processing Signals

- Originally almost all electronics performed analog signal processing
 - Used standard components such as resistors, capacitors, transistors
- Analog signal processing is often complicated
 - Usually requires custom hardware
 - Designing analog signal processing circuitry takes time and therefore is expensive
- Specialized application, like fail-safe systems, require analog signal processing
- For many general applications it is beneficial to move the processing onto a digital platform like a microcontroller
 - Multiple different signal processing operations can be performed by the same hardware
 - It is easy to modify the processing by modifying the code
- Since most real-world signals are analog, we must convert them into digital signals before we can process them using a microcontroller
 - When converting analog signals to digital signals we lose some information



Signals in Your Project



- The smart energy monitor needs to measure voltage and current to determine power drawn by the load
- Voltage and current are analog electrical signals and therefore they need to be converted into digital signals in order to use your microcontroller to perform signal processing
- We will use the ADC peripheral on the ATmega328P to convert analog electrical signals to digital signals
 - The ADC only accepts analog voltage signals that are in the range between 0V and VCC (e.g. 5V)
 - Because of this you designed circuitry to convert the voltage and current you wish to measure into two analog voltage signals that falls within this range



THE UNIVERSITY OF
AUCKLAND
Te Whare Wānanga o Tāmaki Makaurau
NEW ZEALAND

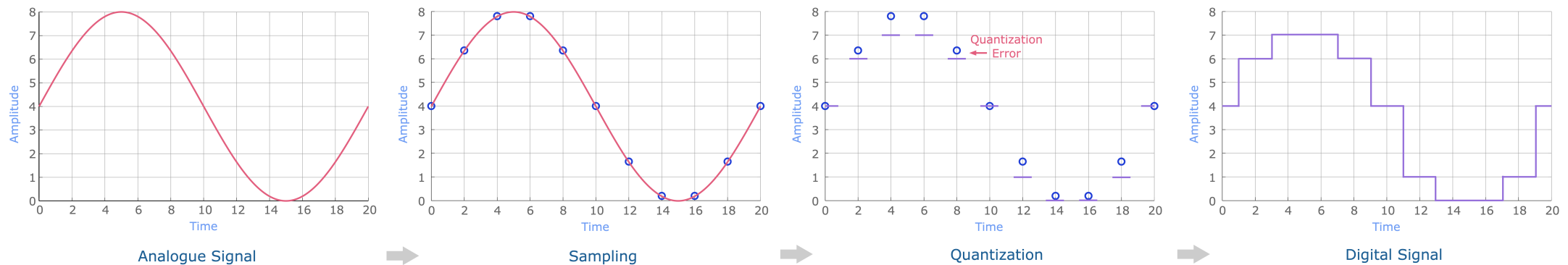
Analog to Digital Converters

Fundamentals Principles



Analog to Digital Conversion

- An analog to digital converter (ADC) converts an analog signal to a digital one
- We know that an analog signal is continuous in both time and amplitude, whereas a digital signal is not
- Therefore, the analog signal needs to be digitized/discretized
 - A process known as sampling is used to convert the analog signal into discrete time steps
 - A process known as quantization is used to convert the analog into discrete amplitudes
- The analog to digital conversion process introduce error
 - For example during quantization information is lost





The Sampling Process

- The size of the time intervals the signal is split into, referred to as sample time, is dependent on the sampling frequency (also called sampling rate), measured in hertz

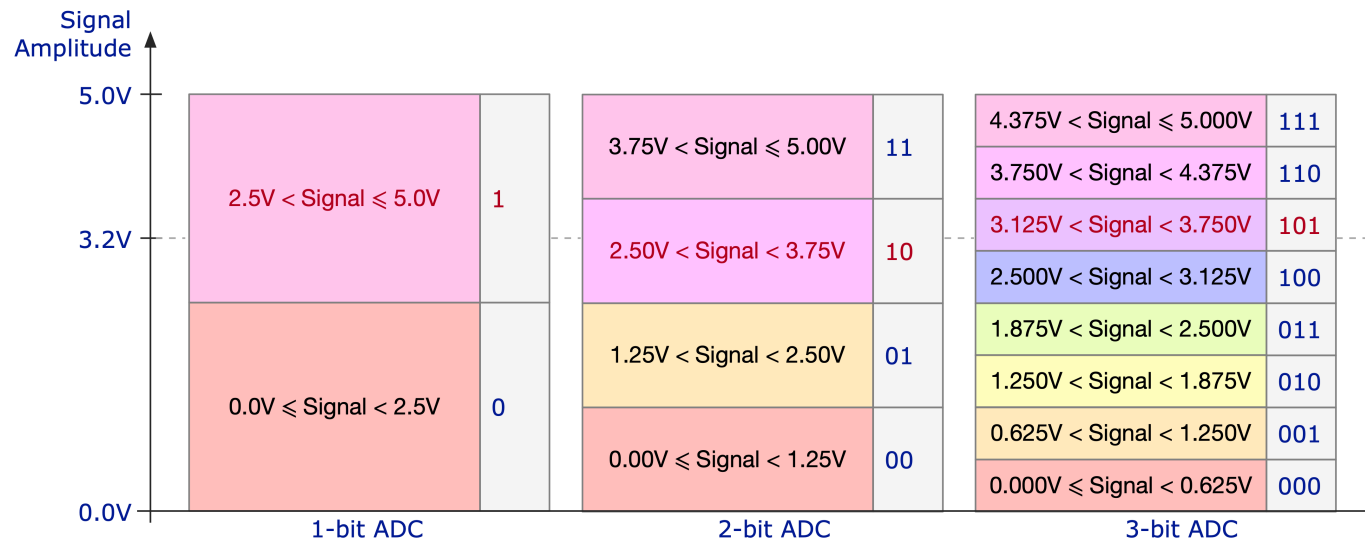
$$t_{sample} = \frac{1}{f_{sample}}$$

- Higher sampling rates allow the original analog signal to be captured more accurately
 - However this requires faster ADC hardware, and more memory to store the digital data
- To accurately capture a signal the sampling rate must be greater than the Nyquist frequency
 - Nyquist frequency is equal to twice the highest frequency present in the signal



The Quantization Process

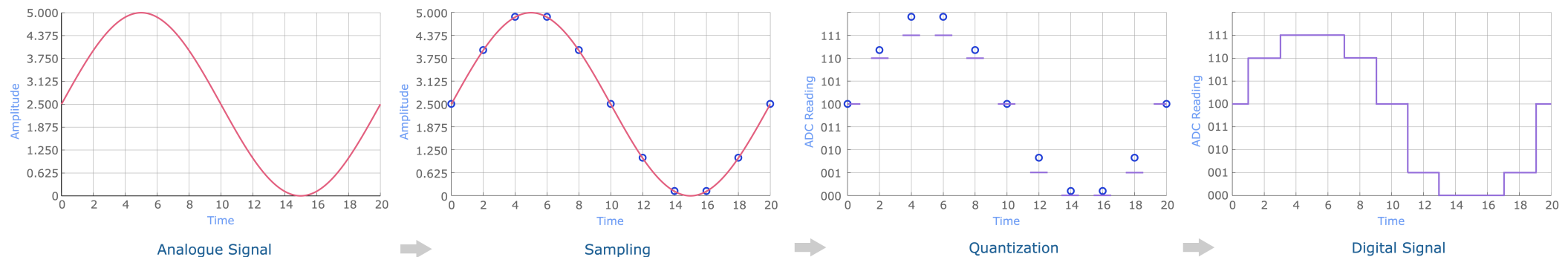
- The full voltage range is broken into discrete intervals
- The number of number of bits available in an ADC to represent the amplitude of a signal determine the number of discrete voltage intervals, as the number of intervals is equal to 2^{bits}
- As an example, lets consider an ADC with a full voltage range of 5V and examine how it will represent a 3.2V analog signal if this ADC had 1-bit, 2-bits and 3-bits to represent an analog signal





A 3-Bit ADC

- Lets examine how a 3-bit ADC would represent a sin wave with an amplitude of 2.5V and an offset of 2.5V
 - Assume the ADC has a full voltage range of 5V
 - The analog signal is sampled every 2ms
 - A 3-bit ADC can break the full voltage range into 8 different intervals (i.e. 2^3)
 - The analog voltage samples are represented by digital numbers depending on which voltage interval each samples falls into (e.g. 3.97V is represented by '110' while 4.9V is represented by '111')
- Higher the bits in the ADC the more accurately we can represent an analog signal





Characteristics of an ADC (PI)

- Acquisition time
 - How long it takes for the ADC hardware to acquire a sample of the analog signal
 - This includes switching multiplexers, charging internal capacitors, waiting for transients to settle, etc.
- Conversion time
 - Time taken by the ADC to convert the sample of the analog signal into a digital number
- Total sampling time
 - This is the total time taken to make a single ADC reading and is therefore the sum of both the acquisition time and conversion time
 - The maximum sampling rate/frequency is limited by the minimum total sampling time

$$f_{sample(max)} = \frac{1}{t_{acquisition(min)} + t_{conversion(min)}}$$

- We will learn more about the acquisition process and the conversion process soon



Characteristics of an ADC (PII)

- Resolution (bits)
 - This is determined by the number of bits available in the ADC to represent the signal
 - For example, an ADC that has a 8-bit resolution got 2^8 amplitude intervals
- Supply voltage (AV_{CC})
 - This is the voltage that is needed to power the internal circuitry of an ADC (e.g. 5V)
- Reference voltage (V_{ref})
 - This specifies the voltage which the input signal is compared to (e.g. 5V) and therefore signals larger than the reference voltage cannot be accurately converted
- Step size (V_{step})
 - This is the smallest signal that can be determined by the ADC and is a function of both the resolution and the reference voltage

$$V_{step} = \frac{V_{ref}}{2^{resolution}}$$



ADC Types

- There are multiple types of ADCs, each focusing on different characteristics
 - For example faster sampling rate, higher resolution, larger dynamic range, temperature stability, lower error, lower cost, etc.
- The most common types include delta-sigma ADC, successive approximation register (SAR) ADC and pipeline ADC
- Microcontrollers predominantly employ successive approximation register (SAR) ADC
 - They have a relatively low part count which keeps cost and size low
 - They have low power consumption
 - They have an adjustable resolution, and generally have mid-range resolution
 - They are generally mid-range for conversion speed
- The ATmega328P has an SAR ADC



Example: Sampling Rate and Resolution

An ADC is used to convert an analog signal that is between 0V and 3V to a digital signal. This ADC has a minimum acquisition time of 140ns, and a minimum conversion time of 360ns.

- What is the maximum sampling rate for the ADC?
- What is the maximum frequency that can be present in a signal such that the ADC could still accurately measure the signal?
- What should be the minimum reference voltage that could be used in this design?
- What should be the minimum supply voltage this ADC should be specified to?
- If the ADC should be capable of measuring down to a 10mV signal, what should be the minimum resolution of the ADC? Assume the ADC uses the minimum reference voltage needed for this design.



THE UNIVERSITY OF
AUCKLAND
Te Whare Wānanga o Tāmaki Makaurau
NEW ZEALAND

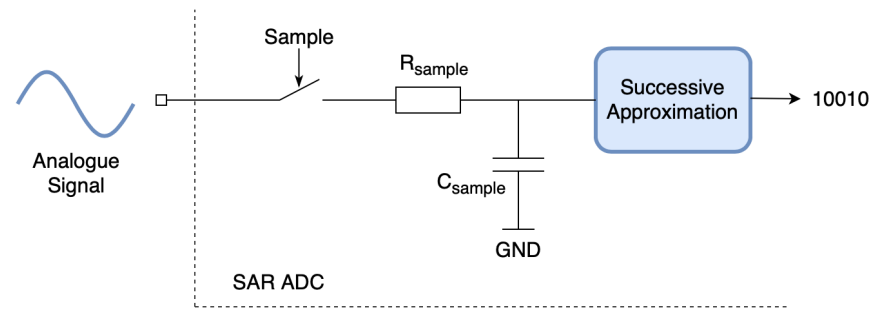
SAR ADC

Operating Principles



The Sampling Process (PI)

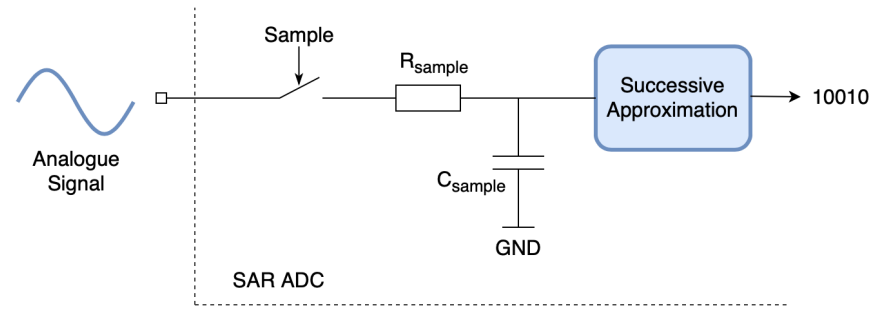
- Recall that we want to take samples of the analog signal at discrete points in time
- This is because the quantization process takes time as it is not an instantaneous process
 - During the quantization process we need to keep the signal being converted constant
- This is achieved by using a sample and hold circuit to capture a “snapshot” of the analog signal and store it until the quantization process is completed
 - The characteristics of the sample and hold circuit are important when designing the analog circuitry that produce the input signal to the ADC
 - Ideally the input to the ADC should be coming from a low impedance source





The Sampling Process (PII)

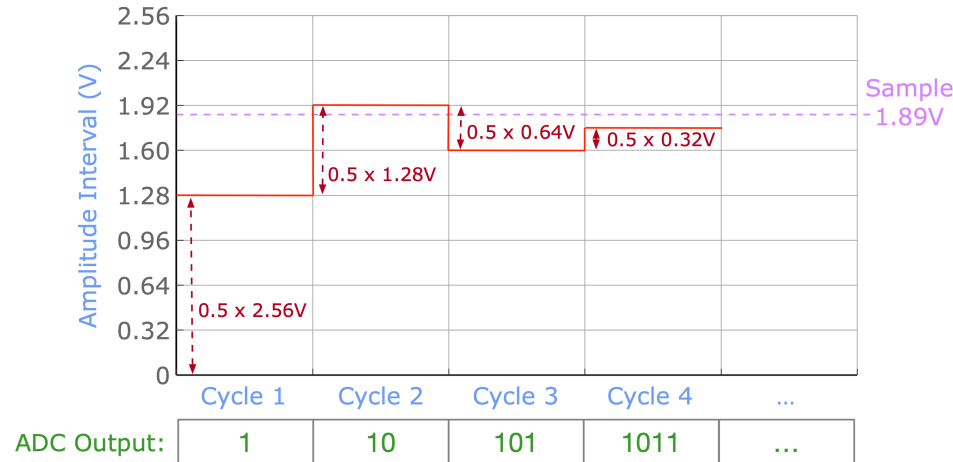
- At the start of the conversion the capacitor is connected to the input signal by closing the switch in order to charge it to the input voltage
 - The length of time the capacitor is charging for is the acquisition time
- Once charged the capacitor is disconnected by turning the switch off
- The quantization is done on the capacitor voltage
 - Any changes in the input signal does not impact the quantization process as switch is off
- The length of time taken for this quantization process is the conversion time
 - After completing an ADC conversion a new sample is taken and the process repeats





The Quantization Process

- The sample of the analog signal is converted to a digital number over a series of ADC clock cycles
 - Hence the name “successive approximation”, an algorithm that dates back to the 1500s
- At each cycle, the sample of the analog signal is compared to an internally generated voltage that is half the remaining amplitude interval (starts with half the value of V_{ref})
 - If sample higher than internal voltage, output bit of this cycle is set to 1, otherwise 0
 - As an example lets see how an ADC with a V_{ref} of 2.56V converts a 1.89V sample





SAR Conversion in Tabular Format

- Lets examine how an 8-bit SAR ADC that has a $2.56V V_{ref}$ converts a sample of an analog signal that has an amplitude of $1.89V$.
 - Has 256 amplitude intervals and the step size is $10mV$ (i.e. $2.56/256$)
- Result of the conversion is $10111101b$ (i.e. 189) which equates to $1890mV$ (i.e. $189 \times 10mV$)

Bit	Upper	Lower	Middle	Comparison	Set	7	6	5	4	3	2	1	0
Cycle 1	256	0	128	$128 \times 10mV = 1.28V$	Yes	1	X	X	X	X	X	X	X
Cycle 2	256	128	192	$192 \times 10mV = 1.92V$	No	1	0	X	X	X	X	X	X
Cycle 3	192	128	160	$160 \times 10mV = 1.60V$	Yes	1	0	1	X	X	X	X	X
Cycle 4	192	160	176	$176 \times 10mV = 1.76V$	Yes	1	0	1	1	X	X	X	X
Cycle 5	192	176	184	$184 \times 10mV = 1.84V$	Yes	1	0	1	1	1	X	X	X
Cycle 6	192	184	188	$188 \times 10mV = 1.88V$	Yes	1	0	1	1	1	1	X	X
Cycle 7	192	188	190	$190 \times 10mV = 1.90V$	No	1	0	1	1	1	1	0	X
Cycle 8	190	188	189	$189 \times 10mV = 1.89V$	Yes	1	0	1	1	1	1	0	1



Acquisition Time & Conversion Time

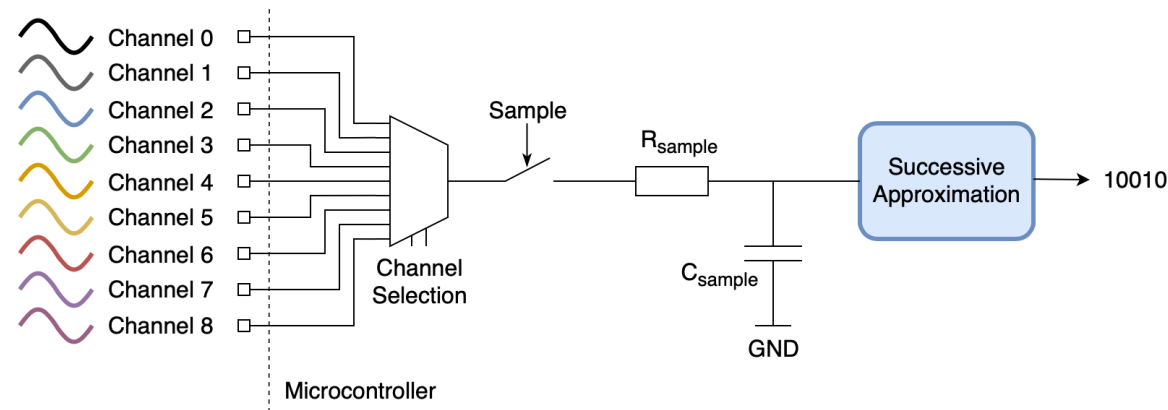
- The acquisition time must be long enough to allow the sample and hold capacitor to charge
 - This is dependent on the source impedance (R_{signal}) of the signal being measured
 - The capacitor is fully charged after approximately 5 time constants (τ)

$$\tau = (R_{\text{signal}} + R_{\text{sample}})C_{\text{sample}}$$

- The conversion time is dependent on the resolution and the cycle time
 - One cycle is required per bit of resolution
 - Shorter cycle times will result in faster conversions, but the minimum cycle time will be limited by the ADC hardware capability, as well as the clock speed of the MCU
- As the ADC interacts with the microprocessor inside the MCU its clock cycles must be aligned
 - The ADC is almost always clocked by the same clock as the microprocessor, or a scaled derivative
 - The ADC unit of the ATmega328P requires a clock frequency **between 50kHz and 200kHz** (operating outside of this range impacts the ADC accuracy)

Multiple ADC Channels

- When using an MCU, often more than one analog signal needs to be converted
- Rather than having multiple ADC units connected to a single pin each, a single ADC unit is often used
 - The pins are connected to the ADC unit via a multiplexer
- The inputs are called “ADC channels”
- Because signals are multiplexed, samples taken from each signal will not be at the same point in time
 - If 2 signals need converting, a sample of the 2nd signal can only be taken after converting the 1st
 - Some MCUs have 2 or more ADC units to allow simultaneous sampling of 2 or more signals





THE UNIVERSITY OF
AUCKLAND
Te Whare Wānanga o Tāmaki Makaurau
NEW ZEALAND

The ADC on ATmega328P

Configuring and Using



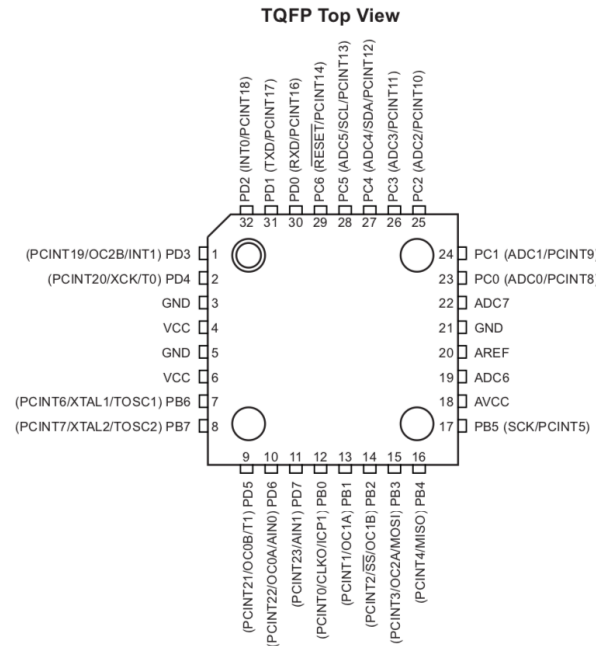
ATmega328P ADC Peripheral

- 10-bit resolution
- 65us – 260us conversion time
- Up to 15k samples per second at full resolution
- 0V to VCC input voltage range
- Three options for reference voltage
 - Internal 1.1V reference voltage
 - ADC supply voltage (AV_{CC})
 - Externally generated reference voltage
- 8 multiplexed channels
- Single conversion or continuous conversion modes
- Interrupt on conversion completion
- ± 2 LSB absolute accuracy



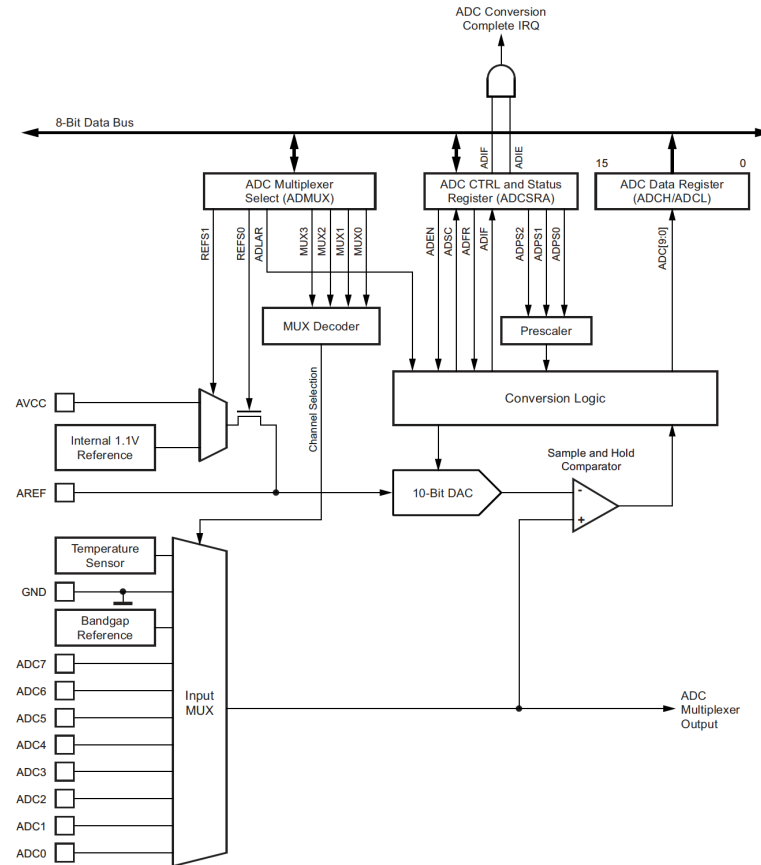
ATmega328P ADC Pins

- In the ATmega328P there are 8 ADC pins (PC0 to PC5 together with ADC6 and ADC7)
 - By default these pins are configured as analog inputs (this is not always the case)
- Though AIN0 and AIN1 also accept analog inputs they are used for the comparator





ATmega328P ADC Implementation





ATmega328P ADC Registers

- The ATmega328P **only has one ADC** and this is named ADC
 - The 8 ADC channels (ADC0 to ADC7) are multiplexed
- There are 5 registers associated with the ADC

Register	Functionality
ADMUX	ADC Multiplexer Selection Register
ADCSRA	ADC Control and Status Register A
ADCSRB	ADC Control and Status Register B
DIDR0	Digital Input Disable Register
ADCH & ADCL	ADC Data Registers

- ADMUX allows selecting the channel to convert and the ADC data registers contain the conversion result
- The remaining three registers are used to control the peripheral
 - Some fields are set by the user to control the operation
 - Some fields are set by the ADC to indicate its current state



ADMUX Register

Bit	7	6	5	4	3	2	1	0
ADMUX	REFS[1..0]		ADLAR	-	MUX[3..0]			
Read/Write	R/W	R/W	R	R/W	R/W	R/W	R/W	R/W
Reset Value	0	0	0	0	0	0	0	0

- REFS[1..0]: *Reference Selection* where 00 = AREF, 01 = AVCC, 10 = Reserved, 11 = Internal 1.1V
- ADLAR: *Left Adjust Result*
- MUX[3..0]: *Channel Selection*

MUX3..0	Single Ended Input
0000	ADC0
0001	ADC1
0010	ADC2
0011	ADC3
0100	ADC4
0101	ADC5
0110	ADC6
0111	ADC7
1000	ADC8 ⁽¹⁾
1001	(reserved)
1010	(reserved)
1011	(reserved)
1100	(reserved)
1101	(reserved)
1110	1.1V (V _{BB})
1111	0V (GND)



ADCSRA Register

Bit	7	6	5	4	3	2	1	0
ADCSRA	ADEN	ADSC	ADATE	ADIF	ADIE	ADPS[2..0]		
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset Value	0	0	0	0	0	0	0	0

- ADEN: *ADC Enable*
- **ADSC**: *Start Conversion* bit needs to be set to start each conversion in single conversion mode while in free running mode this bit only needs to be set for the first conversion
- ADATE: *Auto Trigger Enable* allows starting an ADC conversion using a trigger signal
- **ADIF**: *Interrupt Flag* is set after completing a conversion and cleared when executing the corresponding interrupt
- ADIE: *Interrupt Enable*
- ADPS[2..0]: *Prescaler Select* where 000 = 2, 001 = 2, 010 = 4, 011 = 8, 100 = 16, 101 = 32, 110 = 64, 111 = 128



ADCSRB Register

Bit	7	6	5	4	3	2	1	0
ADCSRB	-	ACME	-	-	-	ADTS[2..0]		
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset Value	0	0	0	0	0	0	0	0

- *ACME: Analog Comparator Multiplexer Enable*
- *ADTS[2..0]: Auto Trigger Source* when ADATE bit is set, these bits selects which source will initiate an ADC conversion

ADTS2	ADTS1	ADTS0	Trigger Source
0	0	0	Free running mode
0	0	1	Analog comparator
0	1	0	External interrupt request 0
0	1	1	Timer/Counter0 compare match A
1	0	0	Timer/Counter0 overflow
1	0	1	Timer/Counter1 compare match B
1	1	0	Timer/Counter1 overflow
1	1	1	Timer/Counter1 capture event



DIDRo Register

Bit	7	6	5	4	3	2	1	0
DIDR0	-	-	ADC5D	ADC4D	ADC3D	ADC2D	ADC1D	ADC0D
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Reset Value	0	0	0	0	0	0	0	0

- **ADCxD: *Digital Input Disable*** when ADC0-ADC5 pins are used as analog inputs the digital input buffer on the corresponding pin can be disabled by setting the ADCxD bit to save power (ADC6 & ADC7 do not have buffers)



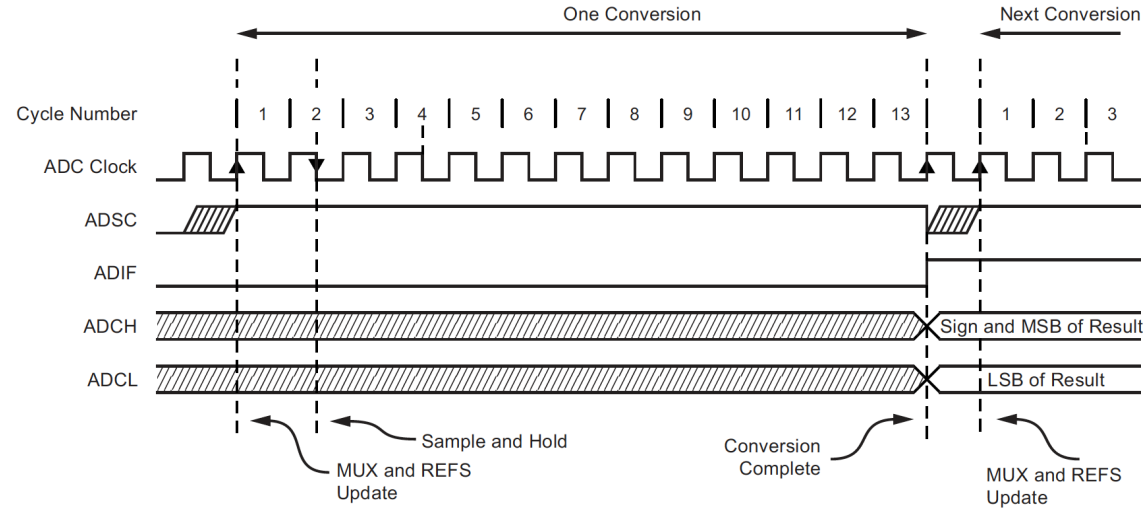
ADCL and ADCH Registers

Bit	7	6	5	4	3	2	1	0
ADCH	ADC[9..8]							
ADCL	ADC[7..0]							
Read/Write	R	R	R	R	R	R	R	R
Reset Value	0	0	0	0	0	0	0	0

- ADC[9..0]: *ADC Result* registers are updated at the end of each conversion and to obtain the results (when ADLAR is cleared) read ADCL followed by ADCH (alternatively you can directly read the register ADC and assign it to a 16-bit variable)



ADC Timing Diagram



- The input signal is sampled 1.5 cycles after the conversion is started in normal conversion mode
 - In auto triggered mode sample and hold takes 2 cycles
- The digitization of the sampled signal takes 11.5 cycles
 - The total conversion takes 13 cycles in normal conversion mode and 13.5 cycles in auto triggered conversion mode

Configuring the ADC

- First we need to configure the ADC as per our needs
 - Need to set the bits of the [ADMUX](#), [ADCSRA](#), [ADCSRB](#) and [DIDRO](#) registers
 - Do this in an initialization function
- In the lectures, as an example, we will configure the ADC to operate in the single conversion mode
 - For the project you are required to operate the ADC in the auto triggered mode using a timer as the trigger source

```
// This function configures the ADC to use AVCC as reference and read ADC0 in single conversion mode
void adc_init(void){
    ADMUX = 0b01000000;    //AVCC set as reference, ADC0 selected and results are right adjusted
    ADCSRA = 0b10000100;   //Set ADEN bit to 1 (enable ADC) and prescaler to 100 (i.e. 16)
                           //To improve readability write in the form "ADCSRA |= (1<<ADEN) | (4<<ADPS0);"
    ADCSRB = 0b00000000;   //Using the single conversion mode (for project you have to use auto trigger mode)
    DIDRO = 0b00000000;    //Buffers are not disabled (for project you may disable the buffers to save power)
}
```



Performing a Conversion

- We also know how to use the ADCL & ADCH registers to read the conversion results
 - Select the input channel and then start a conversion by setting ADSC bit
 - Wait for the conversion to finish by polling the ADIF bit and read result from ADCL & ADCH
 - We typically do the above in a function dedicated to read a given channel

```
// This function reads an ADC channel and return results
uint16_t adc_read_channel_single_conversion(uint8_t channel){
    ADMUX &= 0xF0;           //Clear channel selection
    ADMUX |= channel;        //Set the channel to convert

    ADCSRA |= (1 << ADSC);   //Starting an ADC conversion

    while ((ADCSRA & (1 << ADIF)) == 0) { //ADIF bit is checked to see if it is 0
        ;                               //If ADIF bit is not 1, wait until it becomes 1
    }

    return ((ADCL << 0) | (ADCH << 8)); //Alternatively you can write return ADC;
}
```



Completing the ADC Code

```
#define F_CPU 2000000UL           //Defining system clock frequency

#include <avr/io.h>               //Needed for using the macros for register addresses
#include <stdio.h>                //Needed for printf()
#include <util/delay.h>           //Needed for using _delay_ms()
#include "uart.h"                //Including our uart peripheral library
#include "adc.h"                 //Including our adc peripheral library

static FILE usart_stdout = FDEV_SETUP_STREAM(uart_printf, NULL, _FDEV_SETUP_WRITE); //Creating a stream variable

int main(void){
    stdout = &usart_stdout;      //Binding the steam variable to stdout
    uart_init(9600);             //Initializing the UART to 9600 baud, 8N1, with no parity
    adc_init();                  //Initializing the ADC
    uint32_t dc_voltage_mv = 0;  //Initializing variable to hold ADC result

    while (1){
        dc_voltage_mv = ((uint32_t)adc_read_channel_single_conversion(2) * 5000)/1024; //Converting result to mV
        printf("Voltage: %d mV\n\r", (uint16_t)dc_voltage_mv); //Printing the result
        _delay_ms(500); //Repeat every 0.5s
    }
}
```



Errors During ADC Conversion

- So far we considered an ideal ADC
 - In reality the ADC conversion results has offset error, gain error, quantization error, integral non-linearity and differential non-linearity
- The absolute accuracy is the sum of the error
 - It is measured in Least-Significant-Bits (LSB)
- Given that one LSB is equal one step size, we can determine how many volts the converted reading might differ from the actual value

$$V_{\text{Absolute Error}} = V_{\text{step}} \times LSB_{\text{Absolute Error}}$$

$$V_{\text{ADC}} - V_{\text{Absolute Error}} \leq V_{\text{Actual}} \leq V_{\text{ADC}} + V_{\text{Absolute Error}}$$



Things To Note

- Your ADC function is blocking and your MCU cannot do anything while taking an ADC reading
 - For the project you will have to use an ADC interrupt
 - The ADC also need to be triggered based on a timer event to take an ADC reading every 0.1ms
- You must understand examples you use for the lab or the project
 - Your individual understanding will be assessed during all assessments
 - During tests you will have to develop a C program yourself and simulate it on Proteus
 - All team members must therefore understand every part of the design
- It is critical that you complete the pre-lab of the ADC lab before your lab session so that you can get help with the later parts of the lab
 - Please do not waste the opportunity to get support in the labs
 - With many students in the class, closer to due dates, there will be limited opportunities to ask for help



THE UNIVERSITY OF
AUCKLAND
Te Whare Wānanga o Tāmaki Makaurau
NEW ZEALAND

Acknowledgments

These slides are adapted from material prepared by Travis Scott & Muhammad Nadeem